PreJud

Manual Técnico, Arquitetura e Modelo Operacional

Documentação oficial para onboarding técnico, arquitetura do sistema, domínio Agreement, reputação e evolução do produto

Versão 1.0 | Documentação Técnica Corporativa

Plataforma SaaS para Formalização Digital de Relações Profissionais

Sumário

1. Visão Geral do Produto
    - 1.1 O que é o PreJud
    - 1.2 Problemas que Resolve
2. Papéis do Sistema
3. Entidade Central: Agreement
4. Arquitetura do Frontend
    - 4.1 Estrutura de Pastas
    - 4.2 Papel de cada Camada
5. Mapa de Rotas
6. State Machine do Agreement
7. Timeline do Caso
8. Componentes Principais
9. Services, Hooks e Domínio
10. Sistema de Reputação
11. Modelagem do Banco de Dados
12. Regras para Desenvolvedores
13. Riscos Arquiteturais e Cuidados
14. Ordem Recomendada de Evolução
15. Boas Práticas para Novos Devs
16. Mapa Rápido de Arquivos Críticos
17. Conclusão

1. Visão Geral do Produto

1.1 O que é o PreJud

O **PreJud** é uma plataforma SaaS para formalização digital de relações profissionais entre freelancers e clientes. Ele transforma relações informais em acordos rastreáveis, documentados e verificáveis, registrando tudo em uma timeline cronológica do caso.

A plataforma registra:

- Criação do acordo
- Envio do convite ao cliente
- Confirmação ou contestação
- Prorrogação de prazo
- Aditivo
- Cobrança
- Notificação
- Disputa
- Encerramento

> **Nota:** O sistema não é apenas uma ferramenta de notificação. Ele é uma infraestrutura de relacionamento profissional com trilha verificável.

1.2 Problemas que Resolve

Problema	Solução PreJud
Cliente não paga	Cobrança formalizada com registro de tentativas
Freelancer muda escopo sem registro	Aditivos documentados e aceitos bilateralmente
Prazo muda sem documentação	Prorrogações com aprovação do cliente
Não existe contrato claro	Agreement estruturado com campos obrigatórios
Não existe histórico confiável	Timeline cronológica imutável
Disputas sem prova organizada	Protocolo e hash para verificação
Não há trilha cronológica	Eventos timestampados sequenciais
Não existe reputação bilateral	Sistema de score 0-100 para ambas as partes

2. Papéis do Sistema

Freelancer

Pode realizar as seguintes ações:

- Criar acordo
- Enviar convite ao cliente
- Solicitar prorrogação
- Gerar aditivo
- Criar cobrança
- Emitir notificação
- Encerrar caso

Cliente

Pode realizar as seguintes ações:

- Confirmar acordo
- Contestar acordo
- Aceitar prorrogação
- Propor nova data
- Aceitar aditivo
- Solicitar ajuste
- Contestar cobrança
- Responder notificação
- Marcar "não reconheço este acordo"

3. Entidade Central: Agreement

O **Agreement** é a entidade central do sistema. Representa o acordo entre freelancer e cliente e contém todos os metadados necessários para rastreabilidade.

Estrutura do Agreement

```
interface Agreement {
  id: string;
  title: string;
  freelancerId: string;
  clientId: string;
  serviceType: ServiceType;
  description: string;
  value: number;
  deadline: Date;
  status: AgreementStatus;
  protocol: string;      // Protocolo único do caso
  hash: string;          // Hash de integridade
  timeline: TimelineEvent[];
  createdAt: Date;
  updatedAt: Date;
}
```

Tudo no sistema deriva do Agreement. Todas as ações (prorrogações, cobranças, notificações) estão vinculadas a um Agreement específico.

4. Arquitetura do Frontend

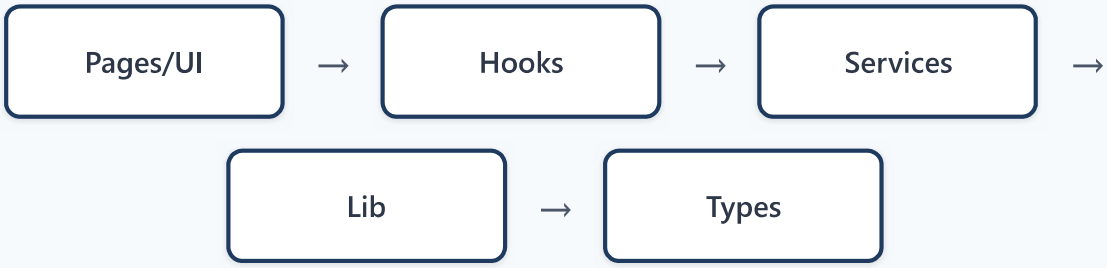
4.1 Estrutura de Pastas

```
frontend/
├── app/                 # Camada de rotas (Next.js App Router)
├── components/          # Componentes reutilizáveis de UI
├── contexts/           # Estado global React
├── hooks/              # Integração entre UI e domínio
├── lib/                # Regras de negócio, permissões e state machine
├── services/           # Acesso a dados e orquestração
├── types/              # Contratos TypeScript do domínio
├── public/             # Assets estáticos
├── package.json
├── tsconfig.json
└── arquivos de configuração
```

4.2 Papel de cada Camada

Camada	Responsabilidade	Exemplo
`app/`	Rotas e páginas do Next.js App Router	`app/dashboard/page.tsx`
`components/`	Componentes reutilizáveis de interface	`AgreementCard.tsx`
`contexts/`	Estado global React (Context API)	`AuthContext.tsx`
`hooks/`	Integração entre UI e lógica de domínio	`useAgreement.ts`
`lib/`	Regras de negócio puras (state machine, permissions)	`lib/agreement/state-machine.ts`
`services/`	Acesso a dados e orquestração de ações	`agreementService.ts`
`types/`	Contratos TypeScript do domínio	`types/agreement.ts`

Diagrama 1 — Fluxo de Arquitetura



5. Mapa de Rotas

Rotas Públicas

Rota	Descrição
`/`	Landing page
`/cliente-nao-pagou-freela`	Página educativa sobre não pagamento
`/modelo-de-contrato`	Templates de contrato
`/como-cobrar-cliente`	Guia de cobrança
`/notificacao-extrajudicial-cobranca`	Informações sobre notificação
`/prova-de-acordo-digital`	Validade jurídica

Autenticação

Rota	Descrição
`/login`	Autenticação de usuário
`/registro`	Cadastro de novo usuário

Área do Freelancer

Rota	Descrição
`/dashboard`	Dashboard principal
`/dashboard/formalizar-acordo`	Criar novo acordo
`/dashboard/acordos/[id]`	Detalhes do acordo

Área do Cliente

Rota	Descrição
`/cliente/dashboard`	Dashboard do cliente
`/cliente/acordos`	Lista de acordos
`/cliente/acordos/[id]`	Detalhes do acordo
`/cliente/cobrancas`	Cobranças recebidas
`/cliente/notificacoes`	Notificações recebidas
`/cliente/historico`	Histórico de interações
`/cliente/convite/[token]`	Aceite de convite

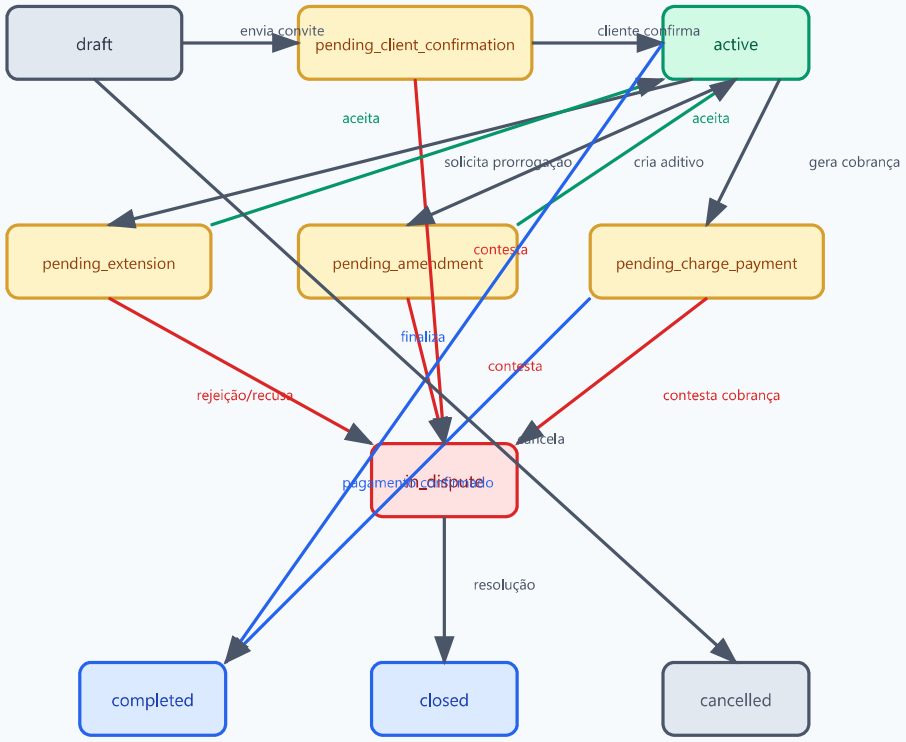
6. State Machine do Agreement

O Agreement possui uma máquina de estados finita que controla todas as transições possíveis. Isso garante consistência e previne estados inválidos.

Estados Possíveis

draft | pending_client_confirmation | active | pending_extension | pending_amendment | pending_charge_payment | in_dispute | completed | cancelled | closed

Diagrama 3 — State Machine do Agreement



Transições Principais

De	Para	Gatilho
draft	pending_client_confirmation	Freelancer envia convite
pending_client_confirmation	active	Cliente confirma
pending_client_confirmation	in_dispute	Cliente contesta
active	pending_extension	Freelancer solicita prorrogação
pending_extension	active	Cliente aceita prorrogação
pending_extension	in_dispute	Cliente rejeita/recusa
active	pending_amendment	Freelancer cria aditivo
pending_amendment	active	Cliente aceita aditivo
pending_amendment	in_dispute	Cliente contesta aditivo
active	pending_charge_payment	Freelancer gera cobrança
pending_charge_payment	completed	Pagamento confirmado
pending_charge_payment	in_dispute	Cliente contesta cobrança

7. Timeline do Caso

A timeline representa a trilha cronológica verificável do relacionamento entre as partes. Cada evento é imutável e possui timestamp.

Diagrama 4 — Sequência de Eventos na Timeline

T+0

agreement_created
Freelancer cria o acordo. Status: draft.

T+1

invitation_sent
Convite enviado ao cliente. Status: pending_client_confirmation.

T+2

client_confirmed
Cliente aceita o acordo. Status: active.

T+N

deadline_extension_requested
Freelancer solicita prorrogação.

T+N+1

deadline_extended
Cliente aceita prorrogação.

T+M

charge_created
Freelancer gera cobrança.

T+M+1

dispute_started
Cliente contesta cobrança. Status: in_dispute.

T+Final

agreement_closed
Caso encerrado após resolução.

Tipos de Eventos

Evento	Ator	Descrição
`agreement_created`	System	Criação inicial do acordo
`invitation_sent`	Freelancer	Envio de convite ao cliente
`client_confirmed`	Cliente	Confirmação do acordo
`client_contested`	Cliente	Contestação inicial
`deadline_extension_requested`	Freelancer	Solicitação de prorrogação
`deadline_extended`	Cliente	Aceite de prorrogação
`amendment_created`	Freelancer	Criação de aditivo
`charge_created`	Freelancer	Geração de cobrança
`notice_sent`	Freelancer	Envio de notificação
`dispute_started`	Cliente/System	Início de disputa
`agreement_closed`	System	Encerramento do caso

8. Componentes Principais

Componentes de Agreement

Componente	Responsabilidade
`AgreementCard`	Visualização resumida do acordo em listas
`AgreementStatusBadge`	Exibição visual do status atual
`BlockedActionsList`	Lista de ações bloqueadas e seus motivos
`Timeline`	Container da timeline de eventos
`TimelineEventItem`	Item individual de evento na timeline

Modais de Ação

Modal	Função
`ProrrogacaoModal`	Solicitar prorrogação de prazo
`AditivoModal`	Criar aditivo ao acordo
`CobrancaModal`	Gerar nova cobrança
`NotificacaoModal`	Enviar notificação formal

9. Services, Hooks e Domínio

Hook Principal: useAgreement

Responsabilidades

- Gerenciar estado local do acordo
- Orquestrar chamadas aos services
- Validar permissões de ação
- Disparar eventos de analytics

```
const {
  agreement,
  loading,
  actions,
  canExtend,
  canCharge,
  transitionState
} = useAgreement(agreementId);
```

Service Principal: agreementService.ts

API do Service

```
export const agreementService = {
  create: (data: CreateAgreementDTO) => Promise<Agreement>,
  getById: (id: string) => Promise<Agreement>,
  sendInvitation: (id: string, clientEmail: string) => Promise<void>,
  confirm: (id: string, clientId: string) => Promise<void>,
  contest: (id: string, reason: string) => Promise<void>,
  requestExtension: (id: string, newDate: Date) => Promise<void>,
  createAmendment: (id: string, data: AmendmentDTO) => Promise<void>,
  createCharge: (id: string, amount: number) => Promise<void>,
  close: (id: string) => Promise<void>,
};
```

Regras de Negócio

State Machine (lib/agreement/state-machine.ts)

Define transições válidas entre estados. Não permite transições inválidas.

Permissions (lib/agreement/permissions.ts)

Define quem pode executar qual ação baseado no papel (freelancer/cliente) e estado atual.

```
export const canExtendDeadline = (
  agreement: Agreement,
  userId: string
): boolean => {
  return agreement.freelancerId === userId &&
         agreement.status === 'active';
};
```

Tipos do Domínio (types/agreement.ts)

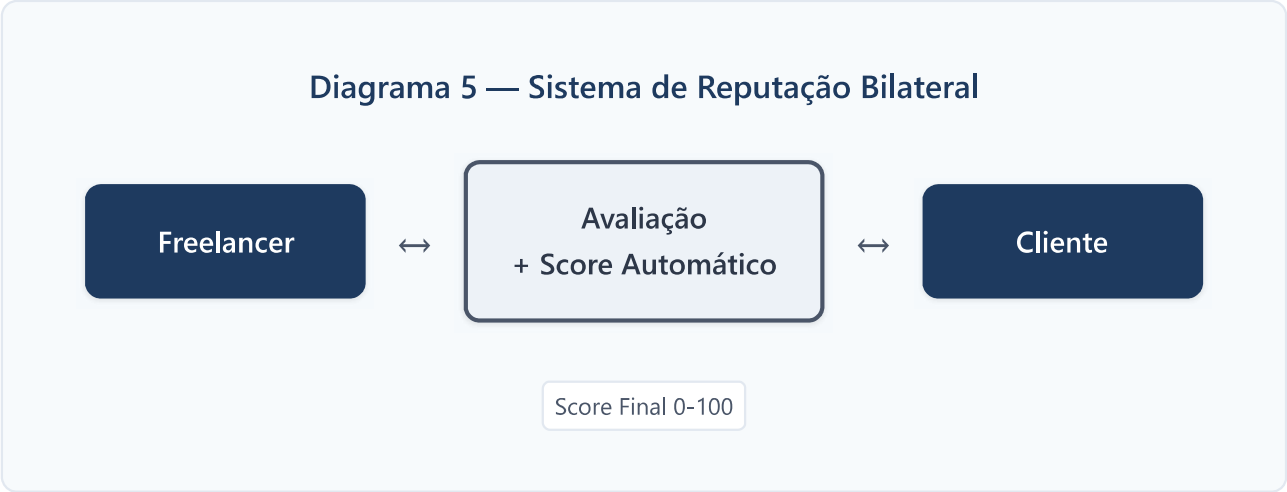
Contratos TypeScript Principais

```
export type AgreementStatus =
  | 'draft'
  | 'pending_client_confirmation'
  | 'active'
  | 'pending_extension'
  | 'pending_amendment'
  | 'pending_charge_payment'
  | 'in_dispute'
  | 'completed'
  | 'cancelled'
  | 'closed';

export interface TimelineEvent {
  id: string;
  type: EventType;
  timestamp: Date;
  actorId: string;
  actorRole: 'freelancer' | 'client' | 'system';
  data: Record<string, unknown>;
  protocol: string;
}
```

10. Sistema de Reputação

O PreJud implementa reputação bilateral (modelo similar ao Uber), onde ambas as partes se avaliam mutuamente.



Escala de Score

0-39	40-59	60-74	75-89	90-100
Alto Risco	Risco Moderado	Regular	Confiável	Excelente

Fatores Positivos

- Acordo concluído com sucesso
- Pagamento confirmado no prazo
- Prazo cumprido conforme acordado
- Avaliação positiva da contraparte
- Resposta rápida às comunicações

Fatores Negativos

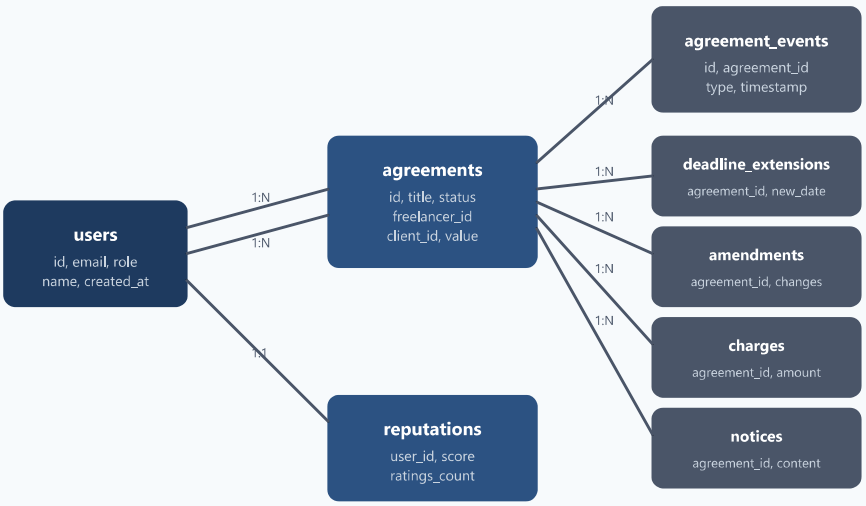
- Disputa iniciada
- Atraso no pagamento
- Não pagamento
- Contestação abusiva
- Não resposta recorrente
- Violação de prazo

Estrutura de Dados

```
interface Reputation {
  userId: string;
  score: number;                    // 0-100
  ratingAverage: number;            // Média de estrelas 1-5
  ratingsCount: number;             // Total de avaliações
  completedAgreements: number;      // Acordos concluídos
  disputedAgreements: number;       // Acordos em disputa
  latePayments: number;             // Pagamentos atrasados
  deadlineViolations: number;       // Violações de prazo
  responseScore: number;            // Pontualidade de resposta
}
```

11. Modelagem do Banco de Dados

Diagrama 6 — Relações entre Entidades



Entidades Principais

Entidade	Descrição	Relações
`users`	Usuários do sistema (freelancer/cliente)	1:N agreements (como freelancer), 1:N agreements (como cliente), 1:1 reputation
`agreements`	Acordo principal entre as partes	N:1 users (freelancer), N:1 users (cliente), 1:N events, 1:N extensions, 1:N amendments, 1:N charges, 1:N notices
`agreement_events`	Timeline do caso	N:1 agreements
`deadline_extensions`	Prorrogações de prazo	N:1 agreements
`amendments`	Aditivos ao acordo	N:1 agreements
`charges`	Cobranças geradas	N:1 agreements
`notices`	Notificações enviadas	N:1 agreements
`reputations`	Reputação consolidada do usuário	1:1 users

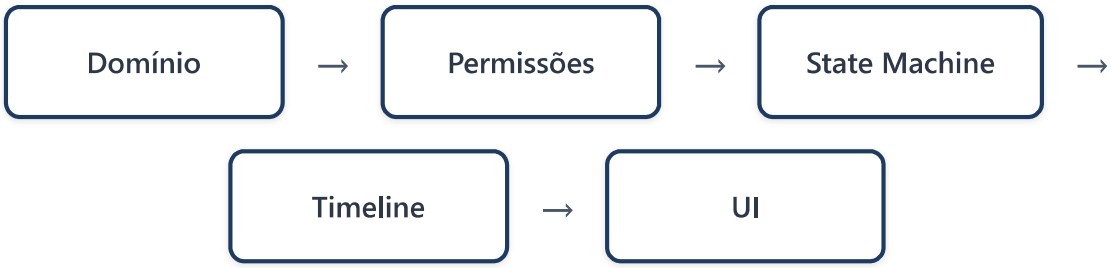
12. Regras para Desenvolvedores

Arquivos Obrigatórios para Consulta: Todo desenvolvedor novo deve consultar primeiro estes arquivos antes de qualquer alteração:

- `types/agreement.ts`
- `hooks/useAgreement.ts`
- `services/agreementService.ts`
- `lib/agreement/state-machine.ts`
- `lib/agreement/permissions.ts`

Fluxo Obrigatório de Desenvolvimento

O sistema deve sempre respeitar a seguinte ordem:



Proibições

NUNCA:

- Criar lógica de domínio diretamente em componentes
- Criar tipos duplicados
- Ignorar a máquina de estados
- Ignorar permissões
- Bypassar a timeline para registrar eventos

13. Riscos Arquiteturais e Cuidados

Risco	Descrição	Mitigação
Lógica paralela	Duplicação de regras de negócio em múltiplos lugares	Centralizar em `lib/`, nunca em componentes
Tipos duplicados	Múltiplas definições de mesma interface	Fonte única de verdade em `types/`
Permissões hardcoded	Verificações de permissão espalhadas no código	Usar `lib/agreement/permissions.ts`
Rotas redundantes	Múltiplas rotas para mesma funcionalidade	Manter mapa de rotas atualizado
Divergência UI/State Machine	Interface permitir ações que a máquina de estados bloqueia	Sempre consultar state machine antes de habilitar ações
Services crescendo	Services acumulando responsabilidades	Separar em services especializados quando necessário
Timeline inconsistente	Estado do agreement e eventos da timeline divergentes	Transações atômicas: atualizar estado + criar evento

14. Ordem Recomendada de Evolução

Para evolução segura do sistema, recomenda-se a seguinte ordem:

1. **Domínio forte** — Consolidar tipos, state machine e permissões
2. **Frontend bilateral** — Finalizar todas as telas de freelancer e cliente
3. **Integração Firestore** — Persistência e sincronização em tempo real
4. **Sistema de reputação** — Cálculo de scores e avaliações

5. **Automações e auditoria** — Notificações automáticas, relatórios, logs

> **Nota:** Cada fase deve estar estável antes de iniciar a próxima. Não pular etapas.

15. Boas Práticas para Novos Devs

Checklist de Entrada no Projeto

- ☐ Ler `types/agreement.ts` completo
- ☐ Entender a state machine em `lib/agreement/state-machine.ts`
- ☐ Revisar `hooks/useAgreement.ts`
- ☐ Estudar diagramas deste manual
- ☐ Configurar ambiente de desenvolvimento
- ☐ Executar testes existentes
- ☐ Fazer primeiro commit apenas de configuração

Padrões de Código

- Usar TypeScript strict mode
- Prefixar interfaces com I apenas quando necessário para clareza
- Manter funções puras em `lib/`
- Hooks devem ter responsabilidade única
- Services não devem conter lógica de UI
- Componentes devem ser "burros" — delegar lógica para hooks

16. Mapa Rápido de Arquivos Críticos

Arquivo	Responsabilidade	Impacto
`types/agreement.ts`	Contratos do domínio Agreement	Alto
`lib/agreement/state-machine.ts`	Máquina de estados e transições	Alto
`lib/agreement/permissions.ts`	Regras de permissão por papel	Alto
`hooks/useAgreement.ts`	Hook principal de gerenciamento	Alto
`services/agreementService.ts`	Orquestração de operações	Alto

`components/AgreementCard.tsx`	Visualização de acordo	Médio
`components/Timeline.tsx`	Exibição de eventos	Médio
`app/dashboard/page.tsx`	Página principal freelancer	Médio
`app/cliente/dashboard/page.tsx`	Página principal cliente	Médio

17. Conclusão

O PreJud é uma plataforma complexa que exige rigor arquitetural para manter consistência e confiabilidade. Este manual técnico serve como referência central para todos os desenvolvedores do projeto.

Lembretes Finais:

- O Agreement é a entidade central — tudo deriva dele
- A state machine é a guardiã da consistência
- A timeline é a fonte da verdade histórica
- O sistema de reputação protege ambas as partes
- Documentação atualizada é tão importante quanto código funcionando

PreJud — Formalizando relações profissionais com trilha verificável.